

Graphs

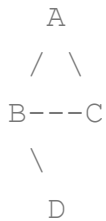
A **Graph** is a non-linear data structure consisting of **vertices (nodes)** and **edges** that connect the vertices.

A graph is represented as:

$$G = (V, E)$$

- **V** = Set of Vertices (Nodes)
- **E** = Set of Edges

Example



Vertices = {A, B, C, D}

Edges = { (A,B), (A,C), (B,C), (B,D) }

Terminology of Graphs

1. Vertex (Node)

A point in a graph.

A, B, C, D

2. Edge

Connection between two vertices.

A ----- B

Edge = (A, B)

3. Adjacent Vertices

Vertices connected directly by an edge.

Example

A ----- B ----- C

- A and B are adjacent.
 - B and C are adjacent.
-

4. Degree

Number of edges connected to a vertex.

Example

 A
 /< \
B----C

- Degree(A) = 2
 - Degree(B) = 2
 - Degree(C) = 2
-

5. Path

Sequence of connected vertices.

Example

$A \rightarrow B \rightarrow D$

Path length = **2**

6. Cycle

A path that starts and ends at the same vertex.

```
A
| \
|  \
B---C
```

Cycle:

$A \rightarrow B \rightarrow C \rightarrow A$

7. Connected Graph

Every vertex can reach every other vertex.

```
A -- B -- C
      |
      D
```

8. Disconnected Graph

Some vertices cannot be reached.

$A -- B$

$C -- D$

Types of Graphs

1. Undirected Graph

Edges have no direction.

A ----- B

Means

A ↔ B

2. Directed Graph (Digraph)

Edges have direction.

A → B

A can go to B only.

3. Weighted Graph

Edges contain weights.

A --5-- B
 \
 2
 \
 C

4. Unweighted Graph

Edges have no weights.

A --- B
|
C

5. Cyclic Graph

Contains at least one cycle.

A
| \
| \
B--C

6. Acyclic Graph

Contains no cycles.

A
|
B
|
C

7. Complete Graph

Every vertex is connected to every other vertex.

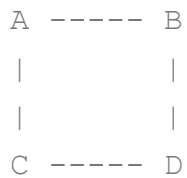
For 4 vertices:

A-----B
| \ / |
| \ / |
| \ / |
| / \ |
| / \ |
| / \ |
C-----D

Adjacency Matrix

Stores a graph using a **2D matrix**.

Graph



Matrix

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

- **1**= Edge exists
- **0**= No edge

Python

```
graph = [  
    [0,1,1,0],  
    [1,0,0,1],  
    [1,0,0,1],  
    [0,1,1,0]  
]  
  
print(graph)
```

Time Complexity

Access Edge

$O(1)$

Space

$O(V^2)$

Adjacency List

Stores neighbors for every vertex.

Graph

```
A  -----  B
|           |
|           |
C  -----  D
```

Representation

```
A → B, C
B → A, D
C → A, D
D → B, C
```

Python

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"]
}
```

```
print(graph)
```

Time Complexity

Access Neighbors

$O(\text{Degree of Vertex})$

Space

$O(V + E)$

Adjacency Matrix vs Adjacency List

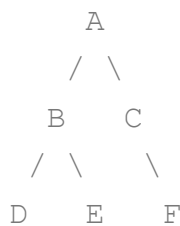
Feature	Adjacency Matrix	Adjacency List
Storage	2D Array	Dictionary/List
Space	$O(V^2)$	$O(V + E)$
Edge Lookup	$O(1)$	$O(V)$ Worst Case
Sparse Graph	Poor	Best
Dense Graph	Best	Good

Breadth First Search (BFS)

BFS visits vertices **level by level**.

Uses a **Queue (FIFO)**.

Example Graph



Start = **A**

Step-by-Step Working

Initial

Queue

[A]

Visited

{ }

Step 1

Remove **A**

Visited

A

Insert neighbors

B C

Queue

[B, C]

Step 2

Remove **B**

Visited

A B

Insert

D E

Queue

[C, D, E]

Step 3

Remove **C**

Visited

A B C

Insert

F

Queue

[D, E, F]

Step 4

Remove **D**

Visited

A B C D

Queue

[E, F]

Step 5

Remove **E**

Visited

A B C D E

Queue

[F]

Step 6

Remove **F**

Visited

A B C D E F

Queue

[]

Traversal

A → B → C → D → E → F

Python Code

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
```

```
def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

bfs(graph, 'A')
```

Output

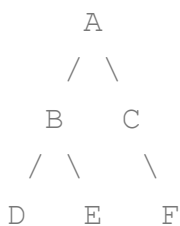
```
A B C D E F
```

Depth First Search (DFS)

DFS explores **as deep as possible** before backtracking.

Uses a **Stack (LIFO)** or **Recursion**.

Example Graph



Step-by-Step Working (Using Stack)

Initial

Stack

[A]

Visited

{ }

Step 1

Pop **A**

Visit

A

Push

C

B

(Stack is LIFO, so **B** is processed first.)

Stack

[C, B]

Step 2

Pop **B**

Visit

A B

Push

E
D

Stack

[C, E, D]

Step 3

Pop **D**

Visit

A B D

Stack

[C, E]

Step 4

Pop **E**

Visit

A B D E

Stack

[C]

Step 5

Pop **C**

Visit

A B D E C

Push

F

Stack

[F]

Step 6

Pop **F**

Visit

A B D E C F

Traversal

A → B → D → E → C → F

Python Code (Recursive)

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = set()

def dfs(node):
    if node in visited:
```

```
        return

    visited.add(node)
    print(node, end=" ")

    for neighbor in graph[node]:
        dfs(neighbor)

dfs('A')
```

Output

```
A B D E C F
```

Python Code (Using Stack)

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def dfs(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()

        if node not in visited:
            visited.add(node)
            print(node, end=" ")

            # Push neighbors in reverse order
            for neighbor in reversed(graph[node]):
                stack.append(neighbor)
```



```
dfs(graph, 'A')
```

Output

```
A B D E C F
```

Applications of BFS

- 1. Finding the shortest path in an unweighted graph.
- 2. Level-order traversal of binary trees.
- 3. GPS/Navigation on unweighted maps.
- 4. Social network friend suggestions.
- 5. Web crawling by search engines.
- 6. Network broadcasting and peer-to-peer systems.

Applications of DFS

- 1. Detecting cycles in graphs.
- 2. Topological sorting of Directed Acyclic Graphs (DAGs).
- 3. Finding connected components.
- 4. Solving mazes and puzzles.
- 5. Backtracking problems (N-Queens, Sudoku, Rat in a Maze).
- 6. Finding paths between nodes.

BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO) / Recursion
Traversal	Level by level	Go deep first
Shortest Path (Unweighted Graph)	Yes	No

Feature	BFS	DFS
Memory Usage	Higher	Lower (typically)
Best For	Shortest Path, Level Traversal	Backtracking, Cycle Detection, Topological Sort
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$	$O(V)$ (Worst Case)